Порівняння desktop- і web-версій TabularDB

Мета та загальний висновок

У TabularDB реалізовано два клієнти для роботи з одними й тими самими табличними даними: настільний застосунок для Windows на базі WPF і вебзастосунок на базі React та TypeScript. Обидва клієнти взаємодіють зі спільним REST API, розробленим на ASP.NET Core. Доменна модель, правила валідації та JSON-сховище реалізовані на серверному боці.

Для подальшого розвитку проєкту доцільно обрати web-клієнт як основний варіант. Він забезпечує ті самі основні можливості, що й desktop-версія, але спрощує розгортання та доступ із різних пристроїв і платформ. Водночас WPF залишається доречним вибором для користувачів, які віддають перевагу окремому Windows-застосунку та інтеграції із системними файловими діалогами.

Це порівняння ґрунтується на реалізованих функціях та архітектурних особливостях обох клієнтів. Окремого тестування продуктивності або дослідження зручності використання не проводилося.

Спільна архітектура та реалізовані можливості

Обидва клієнти підтримують основні операції з базами даних, таблицями та їхнім вмістом. Користувач може створювати й видаляти бази даних і таблиці, визначати схему таблиць, додавати, редагувати та видаляти рядки, а також перейменовувати стовпці й змінювати порядок їх розташування.

Підтримуються такі типи даних: `Integer`, `Real`, `Char` / `Character`, `String`, `Money` і `MoneyInterval`. Первинний ключ може складатися з кількох стовпців, значення яких разом забезпечують унікальність кожного рядка. Для грошових значень застосовуються спільні обмеження точності та діапазону, а грошові інтервали задаються у форматі `мінімум..максимум`.

Сервер на ASP.NET Core надає REST API для обох клієнтів, виконує перевірку даних за допомогою спільної доменної моделі та зберігає каталог баз даних у JSON-файлі. Завдяки цьому обидва інтерфейси працюють за однаковими правилами та використовують спільний формат зберігання даних.

Водночас обидві версії реалізовано за клієнт-серверною архітектурою. Для повноцінної роботи необхідно, щоб сервер API був запущений і доступний клієнту.

Порівняльна таблиця

Критерій	Desktop-версія	Web-версія
Платформа та технології	Windows, .NET 10, WPF, C#, MVVM	Сучасний браузер, React, TypeScript, Vite
Доступ і запуск	Окремий застосунок, який потрібно запустити та налаштувати разом із доступом до API	Доступ через браузер за відповідною адресою; під час розробки Vite запускається окремо від API
Модель даних	Спільні REST API, сервер, доменна модель і JSON-сховище	Спільні REST API, сервер, доменна модель і JSON-сховище
Робота з базами й таблицями	Створення, вибір, видалення та оновлення списків баз даних і таблиць	Аналогічні можливості; перед видаленням відображається підтвердження в браузері
Керування схемою	Типізовані стовпці, складений первинний ключ, перейменування стовпців і зміна їх порядку перетягуванням	Аналогічні можливості; форма створення таблиці також перевіряє введені дані перед надсиланням

Критерій	Desktop-версія	Web-версія
Робота з рядками	Додавання, редагування через форму, видалення та перегляд у табличній сітці	Додавання, редагування, видалення, табличний перегляд і пошук за значеннями
Робота з файлами	Імпорт і експорт JSON за допомогою системних файлових діалогів	Імпорт і завантаження JSON за допомогою можливостей браузера
Зворотний зв'язок	Відображення стану в нижній частині вікна та повідомлення про помилки	Індикатор стану з'єднання, повідомлення про успішні операції та діалог помилки
Залежність від мережі	Потребує доступу до API; локальна автономна робота не передбачена	Потребує браузера та доступу до API; локальна автономна робота не передбачена
Переносимість	Орієнтований на Windows	Працює в сучасних браузерах на підтримуваних настільних платформах
Розвиток інтерфейсу	Використання елементів керування WPF і архітектурного шаблону MVVM	Використання компонентів React і вебстилів; клієнт можна централізовано розгортати

Переваги та обмеження

Desktop-версія

Основна перевага desktop-версії - цілісний інтерфейс окремого Windows-застосунку. WPF добре підходить для роботи з табличними даними на настільному комп'ютері: таблична сітка, системні файлові діалоги, команди та прив'язка даних за допомогою MVVM забезпечують зрозумілу організацію інтерфейсу.

Використання C# також дає змогу працювати з типами доменної моделі на рівні клієнта без необхідності повторно описувати їх іншою мовою програмування.

Водночас desktop-версія має певні обмеження. Вона орієнтована на Windows і потребує окремого встановлення або запуску. Крім того, оскільки застосунок залежить від спільного API, нативний інтерфейс не забезпечує автономної роботи без сервера. Розповсюдження оновлень також потребує оновлення клієнта на відповідних комп'ютерах, що може ускладнювати підтримку кількох робочих місць.

Web-версія

Головна перевага web-версії - доступ через браузер і можливість централізованого розгортання. Користувачам не потрібно встановлювати окремий клієнт на кожному комп'ютері, а оновлення інтерфейсу можна публікувати на сервері без окремого оновлення кожної клієнтської інсталяції.

Додатковими перевагами є пошук рядків за їхніми значеннями та відображення стану з'єднання із сервером. Ці можливості роблять роботу з табличними даними зручнішою. Як і desktop-версія, вебклієнт використовує спільні серверні правила, тому відмінності інтерфейсу не впливають на основну логіку роботи з даними.

Однак web-версія також має обмеження. Для роботи потрібні браузер, мережевий доступ до API та налаштована взаємодія із сервером. Під час розробки необхідно запускати API та інструменти фронтенда окремо. Для використання в робочому середовищі потрібно розгорнути вебклієнт і забезпечити доступність сервера.

Крім того, можливості взаємодії з локальною файловою системою залежать від обмежень браузера. Тому вебклієнт не забезпечує такого самого рівня інтеграції з операційною системою, як окремий настільний застосунок.

Вибір оптимальної технології для TabularDB

Для подальшого розвитку TabularDB доцільно обрати React і TypeScript як основну технологічну платформу клієнтської частини.

Цей вибір зумовлений характером роботи із системою. Основні сценарії передбачають керування базами даних і таблицями, редагування схем, роботу з рядками та обмін JSON-файлами. Для виконання цих завдань можливостей вебінтерфейсу цілком достатньо, а використання браузера спрощує доступ до системи з різних робочих місць.

Вебклієнт також спрощує розповсюдження та підтримку застосунку. Оновлення інтерфейсу можна публікувати централізовано, а для демонстрації проєкту достатньо надати користувачам адресу вебзастосунку за умови доступності API.

Такий вибір узгоджується з поточною реалізацією: web-версія вже підтримує основні операції зі створення, перегляду, редагування та видалення даних, керування схемою таблиць, імпорт і експорт JSON. Пошук за значеннями рядків також розширює можливості роботи з даними через браузер.

Важливо зазначити, що цей висновок не означає, що вебклієнт працює швидше за desktop-версію. Продуктивність обох варіантів окремо не вимірювалася. Вибір web-версії ґрунтується насамперед на доступності, простоті розгортання та зручності подальшої підтримки.

WPF залишається виправданим вибором, якщо система призначена для роботи виключно в середовищі Windows, використовується переважно на одному комп'ютері або потребує тіснішої інтеграції з операційною системою та звичних можливостей окремого настільного застосунку.

Отже, для цього навчального проєкту web-версія є кращим основним варіантом, тоді як desktop-версія може задовольняти потреби користувачів із конкретними вимогами до Windows-інтерфейсу.

Висновок

Обидва клієнти TabularDB реалізують спільні сценарії роботи з табличними даними, використовуючи одне REST API, спільну доменну модель і єдине JSON-сховище. Тому головні відмінності між ними стосуються способу доступу до системи, організації інтерфейсу та особливостей розгортання, а не основних можливостей роботи з даними.

Web-версія є доцільнішим вибором для подальшого розвитку завдяки доступу через браузер, централізованому оновленню та підтримці різних настільних платформ. Desktop-версія зберігає переваги окремого Windows-застосунку та інтеграції із системними можливостями.

Спільна серверна архітектура дає змогу підтримувати обидва клієнти без дублювання основної бізнес-логіки та зберігати однакові правила роботи з даними. Таким чином, вибір між ними залежить насамперед від вимог до доступності, розгортання та середовища використання.